IMPLEMENTASI GAME "CATCH IT FAST!" BERBASIS FPGA MENGGUNAKAN VHDL DAN VGA DISPLAY

MATA KULIAH SISTEM DIGITAL LANJUT

Dosen Pengampu :
Eko Setiawan, S.T., M.T., M.Eng., Ph.D.

DISUSUN OLEH :

REZKY AULIASARIE - 245150300111014
AHMADTOVICH HATHORI ASTRO - 245150300111016
AGATHA TRIOTAMA - 245150300111020
ZEYRA RAHMA ALIVIA - 245150301111016
ALVIANO KEENAN PUTRA RENARDI - 245150307111024
DELVIN NHEAN OLAMINA - 245150307111032

FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA

DAFTAR ISI

BAB I..... 3
PENDAHULUAN..... 3
1.1 Latar Belakang..... 3
1.2 Tujuan..... 3
BAB II..... 4
PERANCANGAN SISTEM..... 4
2.1 Konsep Permainan..... 4
2.2 Arsitektur Top Level..... 4
2.3 Finite State Machine (FSM)..... 4
2.4 Tabel Input & Output..... 6
BAB III..... 7
IMPLEMENTASI DAN PENJELASAN KODE..... 7
3.1 Top Level Module..... 7
3.2 VGA Controller..... 16
3.3 Seven Segment Display..... 18
BAB IV..... 22
HASIL PENGUJIAN..... 22
4.1 Tampilan Visual..... 22
4.2 Logika Permainan..... 24
BAB V..... 25
KESIMPULAN..... 25
DAFTAR PUSTAKA..... 26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi digital memungkinkan implementasi sistem interaktif pada perangkat keras yang dapat diprogram ulang, yaitu *Field Programmable Gate Array* (FPGA). Salah satu aplikasi yang efektif untuk mendemonstrasikan kemampuan pemrosesan paralel dan *real-time* pada FPGA adalah permainan video (*video game*) sederhana.

Game interaktif pada FPGA menuntut integrasi yang presisi antara logika permainan, pembangkitan sinyal video (VGA), dan antarmuka pengguna (input tombol dan output display). Dalam proyek ini, dikembangkan game berjudul "Catch It Fast!". Game ini mensimulasikan permainan menangkap objek buah apel yang jatuh, di mana pemain harus membedakan antara objek yang memberikan poin dan objek yang membahayakan (mengurangi nyawa). Proyek ini dirancang untuk mendemonstrasikan penggunaan VHDL dalam menangani *sprite rendering*, *collision detection*, *random number generation* (LFSR), serta *display multiplexing*.

1.2 Tujuan

1. Membangun sistem permainan interaktif berbasis FPGA menggunakan bahasa deskripsi perangkat keras VHDL.
2. Mengimplementasikan *VGA Controller* untuk menampilkan grafis berwarna (RGB 4-bit) dengan resolusi 640x480.
3. Menerapkan algoritma *Linear Feedback Shift Register* (LFSR) untuk menghasilkan posisi acak (random) jatuhnya objek.
4. Mengimplementasikan logika *sprite* sederhana untuk menampilkan karakter khusus (apel, hati, dan teks) tanpa menggunakan memori eksternal.

BAB II
PERANCANGAN SISTEM

2.1 Konsep Permainan

"Catch It Fast!" dirancang sebagai permainan ketangkasan interaktif di mana pemain mengendalikan sebuah keranjang yang terletak di bagian bawah layar. Mekanisme utama permainan mengharuskan pemain bergerak ke kiri atau ke kanan untuk menangkap objek "Apel Baik" (*Good Apple*) yang memberikan keuntungan berupa tambahan skor sebesar 10 poin. Sebaliknya, pemain harus sigap menghindari "Apel Busuk" (*Bad Apple*) karena jika tertangkap akan memberikan penalti berupa pengurangan satu nyawa dari total tiga nyawa yang tersedia. Untuk memperkaya pengalaman bermain, sistem ini dilengkapi dengan berbagai fitur tambahan, antara lain fitur *Pause* untuk menghentikan permainan sementara waktu, indikator visual sisa nyawa berbentuk ikon hati (*Hearts*), serta elemen latar belakang berupa semak (*bush*) agar tampilan grafis lebih menarik.

2.2 Arsitektur Top Level

Secara struktural, sistem ini dirancang menggunakan pendekatan modular dengan entitas top_level.vhd bertindak sebagai pengintegrasi utama seluruh komponen. Arsitektur ini terdiri dari beberapa blok penyusun utama. Pertama, terdapat *VGA Controller* yang bertugas membangkitkan sinyal sinkronisasi (*HSync* dan *VSync*) serta koordinat piksel (x, y) yang diperlukan monitor untuk menampilkan gambar. Kedua, terdapat *Seven Segment Display Controller* yang berfungsi menampilkan skor permainan dalam format desimal menggunakan teknik *time-division multiplexing* dengan *refresh rate* tinggi untuk menghemat penggunaan pin I/O. Ketiga, terdapat blok *Game Logic* yang tertanam di dalam *Top Level*, mencakup *LFSR* sebagai pengacak posisi jatuhnya apel, *Collision Detector* untuk membandingkan koordinat objek, serta *Sprite Renderer* yang menggambar pola bitmap pada layar.

2.3 Finite State Machine (FSM)

Meskipun sistem ini tidak menggunakan modul FSM eksplisit, logika permainan yang diimplementasikan pada top_level.vhd bekerja berdasarkan konsep state (keadaan) yang diatur oleh sinyal kontrol utama, yaitu game_over dan is_paused. Sistem permainan ini terbagi menjadi tiga state utama dengan karakteristik sebagai berikut:

a. State Play (Bermain) State Play

State Play merupakan keadaan *default* sistem yang aktif segera setelah permainan dimulai atau setelah proses reset dilakukan. Dalam kondisi ini, seluruh logika inti permainan berjalan secara penuh, di mana keranjang dapat digerakkan oleh pemain, apel turun dari atas layar, dan deteksi tabrakan (*collision*) diperiksa secara *real-time*. Keadaan ini berlangsung secara aktif selama sinyal kontrol game_over bernilai '0' dan is_paused bernilai '0'.

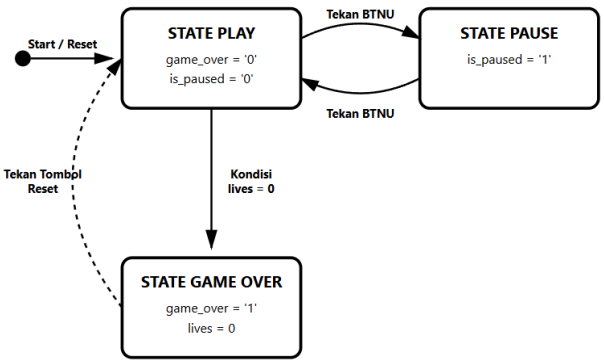
b. State Pause (Jeda)

State Pause adalah mode di mana seluruh pergerakan objek permainan dibekukan sementara waktu, namun tampilan visual pada layar tetap dipertahankan (tidak menjadi gelap) agar pemain dapat melihat posisi terakhir objek. Kondisi ini aktif ketika sinyal is_paused bernilai '1'. Transisi masuk dan keluar dari state ini dipicu oleh penekanan tombol btn_up (BTNU), yang berfungsi sebagai sakelar (*toggle*) untuk menghentikan permainan sejenak dan melanjutkannya kembali ke *State Play* saat ditekan ulang.

c. State Game Over (Permainan Berakhir)

State Game Over merupakan kondisi terminal yang terjadi ketika pemain telah kehabisan kesempatan bermain, yang ditandai dengan variabel lives mencapai angka nol. Pada fase ini, seluruh logika pergerakan objek dihentikan total dan layar akan menampilkan teks "GAME OVER" sebagai indikator visual utama. Secara internal, kondisi ini ditandai dengan aktifnya sinyal game_over (bernilai '1'). Sistem hanya dapat keluar dari keadaan ini melalui penekanan tombol Reset, yang akan memaksa transisi kembali ke *State Play* dengan inisialisasi ulang skor menjadi 0 dan pengisian kembali nyawa.

d. Diagram Finite State Machine (FSM) Game "Catch It Fast!"



2.4 Tabel Input & Output

Nama Port	Tipe	Deskripsi Fungsi
clk	Input	Sinyal clock utama (100 MHz) dari osilator board FPGA.
reset	Input	Tombol (BTNC) untuk mereset skor, nyawa, dan posisi objek ke awal.
btn_left	Input	Tombol (BTNL) untuk menggerakkan keranjang ke arah kiri.
btn_right	Input	Tombol (BTNR) untuk menggerakkan keranjang ke arah kanan.
btn_up	Input	Tombol (BTNU) untuk mengaktifkan fitur Pause/Resume.
vga_r	Output	Sinyal warna merah (4-bit) ke port VGA.
vga_g	Output	Sinyal warna hijau (4-bit) ke port VGA.
vga_b	Output	Sinyal warna biru (4-bit) ke port VGA.
vga_hs	Output	Sinyal sinkronisasi horizontal (Horizontal Sync) ke monitor.
vga_vs	Output	Sinyal sinkronisasi vertikal (Vertical Sync) ke monitor.
seg	Output	Sinyal 7-bit (a-g) untuk mengendalikan segmen display 7-segment.
an	Output	Sinyal 4-bit anoda untuk memilih digit 7-segment yang aktif.

BAB III

IMPLEMENTASI DAN PENJELASAN KODE

3.1 Top Level Module

top_level.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity top_level is
    Port (
        clk       : in STD_LOGIC;
        reset     : in STD_LOGIC; -- BTNC
        btn_left  : in STD_LOGIC; -- BTNL
        btn_right : in STD_LOGIC; -- BTNR
        btn_up    : in STD_LOGIC; -- BTNU (PAUSE)
        vga_r     : out STD_LOGIC_VECTOR(3 downto 0);
        vga_g     : out STD_LOGIC_VECTOR(3 downto 0);
        vga_b     : out STD_LOGIC_VECTOR(3 downto 0);
        vga_hs    : out STD_LOGIC;
        vga_vs    : out STD_LOGIC;
        seg       : out STD_LOGIC_VECTOR(6 downto 0);
        an        : out STD_LOGIC_VECTOR(3 downto 0)
    );
end top_level;

architecture Behavioral of top_level is

    -- Components
    component vga_controller
        Port (
            clk_25mhz : in STD_LOGIC;
            reset     : in STD_LOGIC;
            hsync     : out STD_LOGIC;
            vsync     : out STD_LOGIC;
            video_on  : out STD_LOGIC;
            pixel_x   : out STD_LOGIC_VECTOR(9 downto 0);
            pixel_y   : out STD_LOGIC_VECTOR(9 downto 0)
        );
    end component;
```

```
    component seven_seg_display
        Port (
            clk    : in STD_LOGIC;
            reset : in STD_LOGIC;
            score : in STD_LOGIC_VECTOR(15 downto 0);
            seg    : out STD_LOGIC_VECTOR(6 downto 0);
            an     : out STD_LOGIC_VECTOR(3 downto 0)
        );
    end component;

    -- SPRITE: APPLE (1 = Color, 0 = Transparent)
    type sprite_arr is array (0 to 15) of std_logic_vector(15 downto 0);
    constant APPLE_SPRITE : sprite_arr := (
        "0000000001100000", "0000000001000000", "0000000011100000",
"0000011111110000",
        "0001111111111100", "0011111111111110", "0111111111111111",
"0111111111111111",
        "0111111111111111", "0111111111111111", "0111111111111111",
"0111111111111111",
        "0011111111111110", "0011111111111110", "0001111111111000",
"0000011111100000"
    );

    -- SPRITE: HEART
    constant HEART_SPRITE : sprite_arr := (
        "0000000000000000", "0000000000000000", "0000110000110000",
"0011111001111100",
        "0111111111111110", "0111111111111110", "0111111111111110",
"0011111111111100",
        "0011111111111100", "0001111111110000", "0000111111100000",
"0000011111000000",
        "0000001110000000", "0000000100000000", "0000000000000000",
"0000000000000000"
    );

    -- FONT ROM
    type font_rom_type is array (0 to 127) of std_logic_vector(4 downto 0);
    constant FONT_ROM : font_rom_type := (
        -- 0-9
        "01110", "10001", "10001", "10001", "10001", "10001", "01110",
        "00100", "01100", "00100", "00100", "00100", "00100", "01110",
        "01110", "10001", "00001", "00010", "00100", "01000", "11111",
        "11111", "00010", "00100", "00010", "00001", "10001", "01110",
        "00010", "00110", "01010", "10010", "11111", "00010", "00010",
        "11111", "10000", "11110", "00001", "00001", "10001", "01110",
```

```
    "00110", "01000", "10000", "11110", "10001", "10001", "01110",
    "11111", "00001", "00010", "00100", "01000", "10000", "10000",
    "01110", "10001", "10001", "01110", "10001", "10001", "01110",
    "01110", "10001", "10001", "01111", "00001", "00010", "01100",
    -- G A M E O V E R (Indices 70+)
    "01110", "10001", "10000", "10011", "10001", "10001", "01110", -- G
    "00100", "01010", "10001", "10001", "11111", "10001", "10001", -- A
    "10001", "11011", "10101", "10001", "10001", "10001", "10001", -- M
    "11111", "10000", "10000", "11110", "10000", "10000", "11111", -- E
    "01110", "10001", "10001", "10001", "10001", "10001", "01110", -- O
    "10001", "10001", "10001", "10001", "10001", "01010", "00100", -- V
    "11110", "10001", "10001", "11110", "10100", "10010", "10001", -- R
    "00000", "00000", "00000", "00000", "00000", "00000", "00000", -- Blank
    "01110", "10000" -- Padding
);

signal clk_25mhz : STD_LOGIC := '0';
signal clk_div_counter : integer range 0 to 1 := 0;
signal video_on : STD_LOGIC;
signal pixel_x, pixel_y : STD_LOGIC_VECTOR(9 downto 0);
signal px, py : integer range 0 to 1023;

-- Game objects
signal basket_x : integer range 0 to 639 := 280;
constant BASKET_WIDTH : integer := 60;
constant BASKET_HEIGHT : integer := 15;
constant BASKET_Y : integer := 450;

-- Apples
signal apple_good_x : integer range 0 to 639 := 200;
signal apple_good_y : integer range 0 to 479 := 50;
signal apple_bad_x : integer range 0 to 639 := 400;
signal apple_bad_y : integer range 0 to 479 := 100;
constant APPLE_SIZE : integer := 16;

-- Game state
signal score : unsigned(15 downto 0) := (others => '0');
signal lives : integer range 0 to 3 := 3;
signal game_over : STD_LOGIC := '0';
signal is_paused : STD_LOGIC := '0';
signal btn_up_prev : STD_LOGIC := '0';

-- Logic Counters
signal move_counter : unsigned(19 downto 0) := (others => '0');
signal apple_counter : unsigned(18 downto 0) := (others => '0');
```

```
    signal lfsr : unsigned(9 downto 0) := to_unsigned(731, 10);

    -- Graphics Signals
    signal basket_on : STD_LOGIC;
    signal is_good_apple_pixel : boolean;
    signal is_bad_apple_pixel : boolean;
    signal is_heart_pixel : boolean;
    signal is_text_pixel : boolean;
    signal is_bush_pixel : boolean; -- Sinyal baru untuk Semak

    -- Score digits
    signal score_digit_0, score_digit_1, score_digit_2 : integer range 0 to 9;

begin

    -- Clock Divider
    process(clk)
    begin
        if rising_edge(clk) then
            if clk_div_counter = 1 then
                clk_25mhz <= not clk_25mhz;
                clk_div_counter <= 0;
            else
                clk_div_counter <= clk_div_counter + 1;
            end if;
        end if;
    end process;

    vga_ctrl: vga_controller
        port map ( clk_25mhz => clk_25mhz, reset => reset, hsync => vga_hs, vsync =>
vga_vs,
                   video_on => video_on, pixel_x => pixel_x, pixel_y => pixel_y );

    display: seven_seg_display
        port map ( clk => clk, reset => reset, score => std_logic_vector(score), seg =>
seg, an => an );

    px <= to_integer(unsigned(pixel_x));
    py <= to_integer(unsigned(pixel_y));

    score_digit_0 <= to_integer(score) mod 10;
    score_digit_1 <= (to_integer(score) / 10) mod 10;
    score_digit_2 <= (to_integer(score) / 100) mod 10;

    -- GAME LOGIC
```

```
   process(clk_25mhz, reset)
      variable new_x : integer;
   begin
      if reset = '1' then
         basket_x <= 280;
         apple_good_x <= 200; apple_good_y <= 50;
         apple_bad_x <= 400; apple_bad_y <= 100;
         score <= (others => '0'); lives <= 3; game_over <= '0'; is_paused <= '0';
         lfsr <= to_unsigned(731, 10);
      elsif rising_edge(clk_25mhz) then
         if btn_up = '1' and btn_up_prev = '0' then is_paused <= not is_paused; end if;
         btn_up_prev <= btn_up;

         if game_over = '0' and is_paused = '0' then
            lfsr <= lfsr(8 downto 0) & (lfsr(9) xor lfsr(6));

            -- Move Basket
            move_counter <= move_counter + 1;
            if move_counter(16) = '1' then
               move_counter <= (others => '0');
               if btn_left = '1' and basket_x > 5 then basket_x <= basket_x - 3; end if;
               if btn_right = '1' and basket_x < (640 - BASKET_WIDTH - 5) then
basket_x <= basket_x + 3; end if;
            end if;

            -- Move Apples
            apple_counter <= apple_counter + 1;
            if apple_counter(17) = '1' then
               apple_counter <= (others => '0');
               -- Good Apple (Spawns from Bush y=50)
               if apple_good_y < 479 then apple_good_y <= apple_good_y + 2;
               else apple_good_y <= 50; -- Mulai jatuh dari bawah semak
                  new_x := to_integer(lfsr); if new_x > 620 then new_x := 100; end if;
apple_good_x <= new_x;
               end if;
               -- Bad Apple
               if apple_bad_y < 479 then apple_bad_y <= apple_bad_y + 2;
               else apple_bad_y <= 50;
                  new_x := to_integer(lfsr) + 200; if new_x > 620 then new_x := 300;
end if; apple_bad_x <= new_x;
               end if;

               -- Collisions
               if apple_good_y >= BASKET_Y - APPLE_SIZE and apple_good_y <=
BASKET_Y + 10 then
```

```
                    if apple_good_x >= basket_x - APPLE_SIZE and apple_good_x <=
basket_x + BASKET_WIDTH then
                        score <= score + 10; apple_good_y <= 50;
                        new_x := to_integer(lfsr); if new_x > 620 then new_x := 150; end if;
apple_good_x <= new_x;
                    end if;
                end if;
                if apple_bad_y >= BASKET_Y - APPLE_SIZE and apple_bad_y <=
BASKET_Y + 10 then
                    if apple_bad_x >= basket_x - APPLE_SIZE and apple_bad_x <=
basket_x + BASKET_WIDTH then
                        if lives > 0 then lives <= lives - 1; end if;
                        if lives = 1 then game_over <= '1'; end if;
                        apple_bad_y <= 50;
                        new_x := to_integer(lfsr) + 300; if new_x > 620 then new_x := 400;
end if; apple_bad_x <= new_x;
                    end if;
                end if;
            end if;
        end if;
    end if;
  end process;

  -- GRAPHICS RENDERER

  -- 1. Basket Check
  basket_on <= '1' when (px >= basket_x and px < basket_x + BASKET_WIDTH
and
                      py >= BASKET_Y and py < BASKET_Y + BASKET_HEIGHT)
else '0';

  -- 2. Bush / Semak Check (Background Top)
  process(px, py)
    variable jagged_edge : integer;
  begin
    is_bush_pixel <= false;
    -- Semak ada di Y=0 sampai Y=45 (kira-kira)
    -- Kita buat bagian bawahnya 'jagged' (bergerigi) agar terlihat seperti daun
    -- Polanya menggunakan modulo pixel_x
    jagged_edge := 35 + (px mod 16);

    if py < jagged_edge then
      is_bush_pixel <= true;
    end if;
  end process;
```

```
   -- 3. Sprite Checks (Apples)
   process(px, py, apple_good_x, apple_good_y, apple_bad_x, apple_bad_y)
      variable rel_x, rel_y : integer;
   begin
      is_good_apple_pixel <= false;
      if px >= apple_good_x and px < apple_good_x + 16 and py >= apple_good_y
and py < apple_good_y + 16 then
         rel_x := px - apple_good_x; rel_y := py - apple_good_y;
         if APPLE_SPRITE(rel_y)(15 - rel_x) = '1' then is_good_apple_pixel <= true;
end if;
      end if;

      is_bad_apple_pixel <= false;
      if px >= apple_bad_x and px < apple_bad_x + 16 and py >= apple_bad_y and py
< apple_bad_y + 16 then
         rel_x := px - apple_bad_x; rel_y := py - apple_bad_y;
         if APPLE_SPRITE(rel_y)(15 - rel_x) = '1' then is_bad_apple_pixel <= true;
end if;
      end if;
   end process;

   -- 4. UI Checks (Hearts & Text)
   process(px, py, lives, game_over, score_digit_2, score_digit_1, score_digit_0)
      variable rel_x, rel_y, font_idx : integer;
   begin
      is_heart_pixel <= false; is_text_pixel <= false;
      -- Hearts (Top Right)
      if lives > 0 then
         for i in 0 to 2 loop
            if i < lives then
               if px >= (580 + i*20) and px < (580 + i*20 + 16) and py >= 10 and py <
26 then
                  rel_x := px - (580 + i*20); rel_y := py - 10;
                  if HEART_SPRITE(rel_y)(15 - rel_x) = '1' then is_heart_pixel <= true;
end if;
               end if;
            end if;
         end loop;
      end if;
      -- Score (Top Left)
      if px >= 10 and px < 15 and py >= 10 and py < 17 then -- Digit 2
         rel_x := px - 10; rel_y := py - 10; font_idx := score_digit_2 * 7 + rel_y;
         if FONT_ROM(font_idx)(4 - rel_x) = '1' then is_text_pixel <= true; end if;
      end if;
```

```
    if px >= 18 and px < 23 and py >= 10 and py < 17 then -- Digit 1
       rel_x := px - 18; rel_y := py - 10; font_idx := score_digit_1 * 7 + rel_y;
       if FONT_ROM(font_idx)(4 - rel_x) = '1' then is_text_pixel <= true; end if;
    end if;
    if px >= 26 and px < 31 and py >= 10 and py < 17 then -- Digit 0
       rel_x := px - 26; rel_y := py - 10; font_idx := score_digit_0 * 7 + rel_y;
       if FONT_ROM(font_idx)(4 - rel_x) = '1' then is_text_pixel <= true; end if;
    end if;
    -- Game Over Text
    if game_over = '1' and py >= 240 and py < 247 then
        rel_y := py - 240;
        -- G A M E   O V E R (Logic mapping simplified for brevity)
        if (px>=280 and px<285 and FONT_ROM(70+rel_y)(4-(px-280))='1') or
           (px>=288 and px<293 and FONT_ROM(77+rel_y)(4-(px-288))='1') or
           (px>=296 and px<301 and FONT_ROM(84+rel_y)(4-(px-296))='1') or
           (px>=304 and px<309 and FONT_ROM(91+rel_y)(4-(px-304))='1') or
           (px>=316 and px<321 and FONT_ROM(98+rel_y)(4-(px-316))='1') or
           (px>=324 and px<329 and FONT_ROM(105+rel_y)(4-(px-324))='1') or
           (px>=332 and px<337 and FONT_ROM(91+rel_y)(4-(px-332))='1') or
           (px>=340 and px<345 and FONT_ROM(112+rel_y)(4-(px-340))='1') then
           is_text_pixel <= true;
        end if;
    end if;
end process;

-- 5. FINAL COLOR MIXER (Priority Encoder)
process(video_on, game_over, is_text_pixel, is_heart_pixel, basket_on,
       is_good_apple_pixel, is_bad_apple_pixel, is_bush_pixel)
begin
    if video_on = '1' then
       -- Priority 1: Text (White) - Always on top
       if is_text_pixel then
          vga_r <= "1111"; vga_g <= "1111"; vga_b <= "1111";

       -- Priority 2: Hearts (Red) - Always on top
       elsif is_heart_pixel then
          vga_r <= "1111"; vga_g <= "0000"; vga_b <= "0000";

       -- Priority 3: Game Objects (Basket & Apples)
       elsif basket_on = '1' then
          vga_r <= "1100"; vga_g <= "0110"; vga_b <= "0000"; -- Brown
       elsif is_good_apple_pixel then
          vga_r <= "1111"; vga_g <= "0000"; vga_b <= "0000"; -- Red
       elsif is_bad_apple_pixel then
          vga_r <= "0110"; vga_g <= "0110"; vga_b <= "0110"; -- Gray
```

```
        -- Priority 4: Bush / Semak (Dark Green) - Background Layer 1
        elsif is_bush_pixel then
            vga_r <= "0010"; vga_g <= "1000"; vga_b <= "0010"; -- Forest Green

        -- Priority 5: Sky (Blue) - Background Layer 0
        else
            if game_over = '1' then
                vga_r <= "0011"; vga_g <= "0000"; vga_b <= "0000"; -- Dark Red Sky
(Dead)
            else
                vga_r <= "0100"; vga_g <= "1010"; vga_b <= "1111"; -- Sky Blue
            end if;
        end if;
    else
        vga_r <= "0000"; vga_g <= "0000"; vga_b <= "0000";
    end if;
  end process;

end Behavioral;
```

Penjelasan:

Modul ini berfungsi sebagai otak dari keseluruhan sistem permainan yang menangani berbagai implementasi kunci. A. Pembangkitan Bilangan Acak (LFSR): Untuk menciptakan variasi permainan, posisi jatuhnya apel ditentukan secara acak menggunakan *Linear Feedback Shift Register* (LFSR) 10-bit. Logika ini bekerja dengan menggeser bit dan melakukan operasi XOR pada *taps* tertentu, menghasilkan angka *pseudo-random* yang digunakan sebagai koordinat X saat apel melakukan *respawn*. B. Pengaturan Kecepatan (Timing Counters): Mengingat *clock* bawaan FPGA bekerja sangat cepat (misalnya 100MHz atau 25MHz), pergerakan objek tidak dapat dilakukan pada setiap siklus *clock*. Oleh karena itu, digunakan *counter* sebagai pembagi frekuensi, yaitu move_counter untuk mengatur kecepatan gerak keranjang dan apple_counter untuk mengatur kecepatan jatuh apel agar animasi terlihat wajar bagi pemain. C. Logika Grafis (Priority Encoder): Sistem pewarnaan piksel menerapkan prinsip prioritas tumpukan (*layering*). Jika sebuah koordinat piksel ditempati oleh beberapa objek sekaligus, sistem akan menampilkan objek dengan prioritas tertinggi. Urutannya dimulai dari Teks & UI (Skor, Game Over) di lapisan paling atas, diikuti oleh Ikon Hati, Objek Permainan (Keranjang/Apel), elemen Semak (*Bush*), dan terakhir Langit Biru sebagai latar belakang terbawah. D. Representasi Sprite: Objek visual seperti apel dan hati disimpan dalam memori kode sebagai *Array of Standard Logic Vector*. Logika

rendering akan membaca koordinat relatif piksel; jika bit pada array bernilai '1', maka warna objek akan ditampilkan, sedangkan nilai '0' dianggap transparan.

3.2 VGA Controller

vga_controller.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity vga_controller is
    Port (
        clk_25mhz : in STD_LOGIC;
        reset     : in STD_LOGIC;
        hsync     : out STD_LOGIC;
        vsync     : out STD_LOGIC;
        video_on  : out STD_LOGIC;
        pixel_x   : out STD_LOGIC_VECTOR(9 downto 0);
        pixel_y   : out STD_LOGIC_VECTOR(9 downto 0)
    );
end vga_controller;

architecture Behavioral of vga_controller is
    -- VGA 640x480 @ 60Hz timing
    constant H_DISPLAY : integer := 640;
    constant H_FRONT   : integer := 16;
    constant H_SYNC    : integer := 96;
    constant H_BACK    : integer := 48;
    constant H_TOTAL   : integer := 800;

    constant V_DISPLAY : integer := 480;
    constant V_FRONT   : integer := 10;
    constant V_SYNC    : integer := 2;
    constant V_BACK    : integer := 33;
    constant V_TOTAL   : integer := 525;

    signal h_count : integer range 0 to H_TOTAL-1 := 0;
    signal v_count : integer range 0 to V_TOTAL-1 := 0;
begin

    process(clk_25mhz, reset)
    begin
```

```
        if reset = '1' then
            h_count <= 0;
            v_count <= 0;
        elsif rising_edge(clk_25mhz) then
            if h_count = H_TOTAL-1 then
                h_count <= 0;
                if v_count = V_TOTAL-1 then
                    v_count <= 0;
                else
                    v_count <= v_count + 1;
                end if;
            else
                h_count <= h_count + 1;
            end if;
        end if;
    end process;

    hsync <= '0' when (h_count >= H_DISPLAY+H_FRONT and h_count <
H_DISPLAY+H_FRONT+H_SYNC) else '1';
    vsync <= '0' when (v_count >= V_DISPLAY+V_FRONT and v_count <
V_DISPLAY+V_FRONT+V_SYNC) else '1';
    video_on <= '1' when (h_count < H_DISPLAY and v_count < V_DISPLAY) else
'0';
    pixel_x <= std_logic_vector(to_unsigned(h_count, 10));
    pixel_y <= std_logic_vector(to_unsigned(v_count, 10));

end Behavioral;
```

Penjelasan:

Modul vga_controller ini berfungsi menghasilkan sinyal VGA standar dengan menggunakan clock 25 MHz sebagai pixel clock. Di dalam arsitektur, berbagai constant seperti H_DISPLAY, H_FRONT, H_SYNC, H_BACK, hingga H_TOTAL serta bagian yang mengatur arah perpindahan baris digunakan sebagai acuan timing untuk proses pemindaian horizontal dan vertikal. Perhitungan posisi dilakukan melalui counter h_count dan v_count yang bertambah setiap rising edge dari clk_25mhz, dan keduanya akan di-reset kembali ke nol saat mencapai batas total masing-masing. Sinyal hsync dan vsync ditentukan berdasarkan rentang posisi counter sesuai dengan interval Front Porch, Sync Pulse, dan Back Porch, sementara video_on menandakan apakah koordinat saat ini berada dalam area display aktif. Selanjutnya, nilai pixel_x dan pixel_y dikonversi ke STD_LOGIC_VECTOR untuk menunjukkan posisi pixel yang sedang diproses sehingga modul lain dapat menentukan warna atau output grafis yang akan ditampilkan.

3.3 Seven Segment Display

seven_seg_display.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity vga_controller is
    Port (
        clk_25mhz : in STD_LOGIC;
        reset     : in STD_LOGIC;
        hsync     : out STD_LOGIC;
        vsync     : out STD_LOGIC;
        video_on  : out STD_LOGIC;
        pixel_x   : out STD_LOGIC_VECTOR(9 downto 0);
        pixel_y   : out STD_LOGIC_VECTOR(9 downto 0)
    );
end vga_controller;

architecture Behavioral of vga_controller is
    -- VGA 640x480 @ 60Hz timing
    constant H_DISPLAY : integer := 640;
    constant H_FRONT   : integer := 16;
    constant H_SYNC    : integer := 96;
    constant H_BACK    : integer := 48;
    constant H_TOTAL   : integer := 800;

    constant V_DISPLAY : integer := 480;
    constant V_FRONT   : integer := 10;
    constant V_SYNC    : integer := 2;
    constant V_BACK    : integer := 33;
    constant V_TOTAL   : integer := 525;

    signal h_count : integer range 0 to H_TOTAL-1 := 0;
    signal v_count : integer range 0 to V_TOTAL-1 := 0;
begin

    process(clk_25mhz, reset)
    begin
        if reset = '1' then
            h_count <= 0;
            v_count <= 0;
        elsif rising_edge(clk_25mhz) then
            if h_count = H_TOTAL-1 then
```

```
            h_count <= 0;
            if v_count = V_TOTAL-1 then
               v_count <= 0;
            else
               v_count <= v_count + 1;
            end if;
         else
            h_count <= h_count + 1;
         end if;
      end if;
   end process;

   hsync <= '0' when (h_count >= H_DISPLAY+H_FRONT and h_count <
H_DISPLAY+H_FRONT+H_SYNC) else '1';
   vsync <= '0' when (v_count >= V_DISPLAY+V_FRONT and v_count <
V_DISPLAY+V_FRONT+V_SYNC) else '1';
   video_on <= '1' when (h_count < H_DISPLAY and v_count < V_DISPLAY) else
'0';
   pixel_x <= std_logic_vector(to_unsigned(h_count, 10));
   pixel_y <= std_logic_vector(to_unsigned(v_count, 10));

end Behavioral;
```

Penjelasan:

Modul seven_seg_display ini bertugas menampilkan nilai skor ke layar 7-segment dengan cara mengubah data biner menjadi angka desimal yang mudah dibaca. Dalam prosesnya, nilai skor yang masih berbentuk biner dipecah dulu menjadi digit satuan, puluhan, dan ratusan menggunakan operasi pembagian sederhana. Setelah setiap digit diperoleh, modul menggunakan teknik Multiplexing, yaitu menyalakan setiap digit secara bergantian dengan sangat cepat menggunakan *refresh_counter*. Karena pergantian ini terjadi dalam kecepatan tinggi, mata manusia melihatnya seolah-olah semua digit menyala bersamaan, sehingga angka yang muncul tampak stabil dan jelas pada display.

File constraint (.xdc)

```
## Clock 100MHz
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add
[get_ports clk]
```

```
## VGA Pins
set_property PACKAGE_PIN G19 [get_ports {vga_r[0]}]
set_property PACKAGE_PIN H19 [get_ports {vga_r[1]}]
set_property PACKAGE_PIN J19 [get_ports {vga_r[2]}]
set_property PACKAGE_PIN N19 [get_ports {vga_r[3]}]
set_property PACKAGE_PIN J17 [get_ports {vga_g[0]}]
set_property PACKAGE_PIN H17 [get_ports {vga_g[1]}]
set_property PACKAGE_PIN G17 [get_ports {vga_g[2]}]
set_property PACKAGE_PIN D17 [get_ports {vga_g[3]}]
set_property PACKAGE_PIN N18 [get_ports {vga_b[0]}]
set_property PACKAGE_PIN L18 [get_ports {vga_b[1]}]
set_property PACKAGE_PIN K18 [get_ports {vga_b[2]}]
set_property PACKAGE_PIN J18 [get_ports {vga_b[3]}]
set_property PACKAGE_PIN P19 [get_ports vga_hs]
set_property PACKAGE_PIN R19 [get_ports vga_vs]
set_property IOSTANDARD LVCMOS33 [get_ports vga_*]

## Buttons Control
set_property PACKAGE_PIN W19 [get_ports btn_left]
set_property IOSTANDARD LVCMOS33 [get_ports btn_left]

set_property PACKAGE_PIN T17 [get_ports btn_right]
set_property IOSTANDARD LVCMOS33 [get_ports btn_right]

## Button Pause (BTNU - T18)
set_property PACKAGE_PIN T18 [get_ports btn_up]
set_property IOSTANDARD LVCMOS33 [get_ports btn_up]

## Reset Button (BTNC - U18)
set_property PACKAGE_PIN U18 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]

## 7-Segment Display
set_property PACKAGE_PIN W7 [get_ports {seg[6]}]
set_property PACKAGE_PIN W6 [get_ports {seg[5]}]
set_property PACKAGE_PIN U8 [get_ports {seg[4]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[2]}]
set_property PACKAGE_PIN V5 [get_ports {seg[1]}]
set_property PACKAGE_PIN U7 [get_ports {seg[0]}]
set_property PACKAGE_PIN U2 [get_ports {an[0]}]
set_property PACKAGE_PIN U4 [get_ports {an[1]}]
set_property PACKAGE_PIN V4 [get_ports {an[2]}]
set_property PACKAGE_PIN W4 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports seg*]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports an*]

## Config
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]
```

Penjelasan:

Kode file constraint tersebut berfungsi untuk menetapkan pemetaan antara sinyal-sinyal dengan pin fisik pada FPGA, sekaligus menentukan karakteristik elektris dan konfigurasi dasarnya. Pada bagian Clock 100MHz, perintah set_property PACKAGE_PIN W5 dan IOSTANDARD LVCMOS33 menghubungkan port clk ke pin W5 dengan standar tegangan 3.3V, sementara create_clock mendefinisikan sinyal clock eksternal dengan periode 10ns. Bagian VGA pins memetakan setiap bit dari vga_r, vga_g, dan vga_b serta sinyal sinkronisasi vga_hs dan vga_vs ke pin-pin tertentu, semuanya menggunakan standar I/O LVCMOS33 untuk memastikan kompatibilitas tegangan. Pada bagian Buttons Control dan tombol tambahan seperti btn_up dan reset, masing-masing tombol dihubungkan ke pin fisik yang sesuai agar dapat digunakan sebagai input kontrol. Selanjutnya, bagian 7-Segment Display mengatur pemetaan untuk port seg dan an sehingga sistem dapat mengendalikan tiap segmen dan anoda display. Dan yang terakhir, bagian Config menetapkan konfigurasi global, yaitu CONFIG_VOLTAGE 3.3 dan CFGBVS VCCO, untuk memastikan tegangan konfigurasi dan sumber referensi berada pada level yang sesuai dengan kebutuhan board FPGA.

BAB IV
HASIL PENGUJIAN

4.1 Tampilan Visual

a. Tampilan Awal

Berdasarkan hasil visual yang dilakukan pada tampilan monitor, kami simpulkan bahwa tampilan muncul dan berfungsi dengan baik seperti yang kami rencanakan. Latar belakang langit biru berhasil ditampilkan secara jelas, lengkap dengan dekorasi semak-semak berwarna hijau yang menghiasi atas layar sebagai elemen pendukung suasana agar terlihat natural bahwa apel jatuh dari sebuah pohon. Di bagian bawah terdapat keranjang dengan warna coklat muda yang bekerja sebagaimana mestinya, di mana objek tersebut dapat bergerak secara halus ke kiri maupun ke kanan sebagai respons langsung terhadap input tombol dari pengguna tanpa adanya gangguan atau jeda yang mencolok.



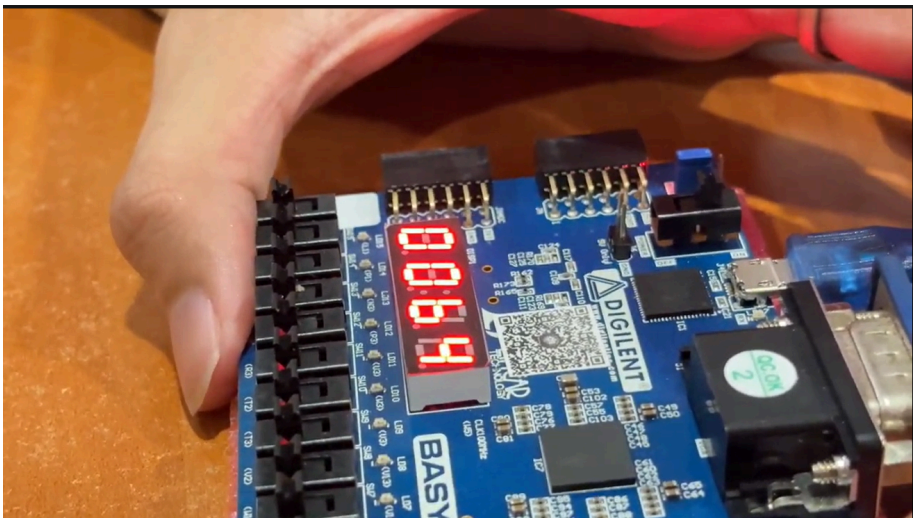
Selain itu, terdapat dua jenis objek yang dirancang untuk jatuh dari bagian atas layar, yaitu Apel Merah sebagai objek yang memberikan poin dan Apel Abu-abu sebagai objek bahaya. Kedua apel tersebut akan muncul sesuai waktu dan kecepatan yang sudah ditentukan. Keduanya terlihat sesuai dengan desain sprite yang telah dirancang sebelumnya, baik dari sisi bentuk, warna maupun perilaku pergerakannya. Secara keseluruhan, tampilan awal program menunjukkan bahwa semua komponen visual dalam permainan mampu berjalan.

b. Tampilan Game Over



Ketika permainan memasuki fase game over, seluruh aktivitas dalam game dihentikan dan tampilan layar berubah sepenuhnya. Latar belakang yang sebelumnya berwarna biru akan berganti menjadi merah sebagai penanda bahwa permainan telah berakhir, disertai teks “Game Over” yang muncul jelas di bagian tengah layar. Pada tahap ini, seluruh elemen interaktif seperti Keranjang di bagian bawah maupun Apel yang biasanya jatuh dari atas layar tidak lagi bergerak. Kondisi tersebut bisa terjadi karena sistem telah berpindah ke state “GAME OVER”, yaitu keadaan dimana semua mekanisme permainan dihentikan dan proses permainan dianggap selesai.

c. Tampilan poin skor pada perangkat Basys-3



Pada program yang sudah kami buat, sistem dirancang untuk menampilkan skor yang diperoleh pemain melalui modul 7-Segment Display pada perangkat Basys-3. Pada gambar ditunjukkan, terlihat tampilan “00b4” pada display tersebut, yang menandakan bahwa pemain sebelumnya berhasil mengumpulkan skor sebesar 180 poin. Nilai ini merupakan hasil konversi dari skor dari dalam permainan ke format yang dapat ditampilkan pada 7-Segment. Selama permainan berlangsung, setiap kali pemain mendapatkan poin tambahan, angka pada display akan langsung diperbarui sehingga menunjukkan perkembangan skor secara real-time. Proses pembaruan ini akan terus berjalan hingga permainan mencapai state “GAME OVER”, sehingga pemain dapat memantau skor yang didapat sepanjang sesi permainan.

4.2 Logika Permainan

Pengujian logika menunjukkan bahwa aturan permainan berjalan dengan akurat. Pada sistem *Scoring*, ketika keranjang menyentuh Apel Merah, skor pada tampilan 7-Segment di display monitor bertambah sebesar 10 poin dengan tampilan bilangan desimal dan skor juga ditampilkan pada 7-Segment Basys3 dengan tampilan bilangan hexadesimal. Sebaliknya, pada mekanisme *Hit/Damage*, ketika keranjang menyentuh Apel Busuk, indikator nyawa berupa ikon hati di pojok kanan atas berkurang satu. Logika *Game Over* juga berfungsi dengan baik; saat ketiga nyawa habis, layar berubah menjadi gelap dan menampilkan teks "GAME OVER". Terakhir, tombol *Pause* terbukti berhasil membekukan pergerakan seluruh objek tanpa mereset status permainan.

BAB V
KESIMPULAN

Berdasarkan seluruh tahapan perancangan, implementasi, hingga pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan utama. Pertama, bahasa VHDL terbukti efektif digunakan untuk membangun logika permainan *real-time* secara murni menggunakan gerbang logika tanpa memerlukan bantuan prosesor (CPU). Kedua, penerapan algoritma *LFSR* 10-bit dinilai cukup memadai untuk memberikan variasi acak pada posisi jatuh objek sehingga permainan tidak terasa monoton. Ketiga, teknik *hardcoded sprite* menggunakan array merupakan solusi efisien untuk menampilkan grafis ikonik sederhana seperti apel dan hati tanpa memerlukan memori eksternal yang kompleks. Terakhir, implementasi sistem prioritas pewarnaan (*Priority Encoder*) menjadi elemen yang sangat krusial dalam manajemen visual VGA untuk menentukan lapisan objek depan dan latar belakang secara tepat.

Projek ini juga menunjukkan bahwa integrasi antara modul tampilan VGA, logika permainan, dan antarmuka perangkat seperti tombol dan 7-Segment Display dapat berjalan dengan sinkron dan stabil meskipun seluruh proses dikendalikan langsung oleh rangkaian digital tanpa sistem operasi maupun perangkat lunak tingkat tinggi. Keberhasilan ini membuktikan bahwa FPGA mampu menjadi platform yang kura untuk membangun sistem interaktif yang responsif. Selain itu, proses perancangan game ini memberi wawasan praktis mengenai bagaimana algoritma, timing, dan manajemen sinyal bekerja secara bersamaan dalam suatu sistem real-time. Secara keseluruhan, projek ini tidak hanya menghasilkan produk permainan yang dapat dimainkan, tetapi juga menjadi pengalaman komprehensif dalam memahami penerapan konsep digital hardware dalam situasi nyata.

DAFTAR PUSTAKA

Digilent Inc. (2016). ***Basys 3 FPGA board reference manual.*** **https://digilent.com/reference/programmable-logic/basys-3/reference-manual**

Horobets, D., & Karimie, A. (2024). ***Catch_the_Apple: VGA Controller using FPGA / VHDL + game*** **[Computer software]. GitHub. https://github.com/Daniil-Horobets/Catch_the_Apple**